

**GHANA COMMUNICATION TECHNOLOGY
UNIVERSITY (GCTU)**

**FACULTY OF COMPUTING AND INFORMATION
SYSTEMS (FOCIS)
DEPARTMENT OF INFORMATION TECHNOLOGY**

**TITLE:
STUDENT ACADEMIC PROGRESS TRACKING SYSTEM**

A Project Work Submitted in Partial Fulfillment of the Requirements For
BSc. Information Technology

BY:

Richard Yawlui (4211231018)
Prince Baffour Awuah (4211230066)
Addo Foster Keteku (4211231161)

SUPERVISOR:

MR. FRANK BOATEN

AUGUST, 2026

DECLARATION

This project is presented as part of the requirements for BSc. Information Technology awarded by Ghana Communication Technology University. We hereby declare that this project is entirely the result of our own hard work, research, and enquiries. We are confident that this project work is not copied from any other person. All sources of information have however been acknowledged with due respect.

AUTHOR: Richard Yawlui

SIGNATURE

STUDENT ID: 4211231018

DATE:

AUTHOR: Prince Baffour Awuah

SIGNATURE

STUDENT ID: 4211230066

DATE:

AUTHOR: Addo Foster Keteku

SIGNATURE

STUDENT ID: 4211231161

DATE:

SUPERVISOR: MR. FRANK BOATEN

SIGNATURE

DATE:

HOD:

SIGNATURE

DATE:

ABSTRACT

University students monitor their academic progress through a scattered collection of notebooks, calculator applications and calendar reminders, none of which communicate with one another. The result is a fragmented record that makes it difficult to establish a current academic standing, to observe performance trends, or to relate study effort to examination outcomes. This project designed and developed a Student Academic Progress Tracking System: a web-based application that consolidates course management, assessment recording, weighted grade and grade point average computation, study session logging, revision through flashcards, document storage and goal tracking into a single interface. Requirements were established through a review of the literature on self-regulated learning, interviews with students and lecturers, and an examination of existing tracking tools; the system was then built using the Waterfall model in HTML5, CSS3 and JavaScript, with Chart.js for visualisation, and is served locally by the Apache web server supplied with XAMPP. All records persist on the user's own device through the Web Storage API and IndexedDB, so no external database server is required and no academic data leaves the machine. Functional, usability and performance testing confirmed that grade calculations are accurate, that data survives browser restarts, and that the interface loads and responds well within the three-second target on both desktop and mobile displays. The system demonstrates that a purpose-built, privacy-preserving and zero-cost academic tracker can give students the integrated overview that general-purpose productivity tools do not provide.

Keywords: academic progress tracking, self-regulated learning, grade point average, data visualisation, client-side storage, XAMPP.

TABLE OF CONTENTS

Declaration.....	ii
Abstract.....	iii
Table of Contents.....	iv
List of Tables.....	vi
List of Figures.....	vii
List of Abbreviations.....	viii
Acknowledgement.....	ix
CHAPTER ONE: INTRODUCTION.....	1
1.1 Background to Study.....	1
1.2 Statement of the Problem.....	2
1.3 Aim and Objectives.....	3
1.4 Significance of the Study.....	3
1.5 Scope of the Study.....	4
1.6 Organization of the Study.....	5
1.7 Conclusion.....	6
CHAPTER TWO: LITERATURE REVIEW.....	7
2.1 Overview of Relevant Literature.....	7
2.1.1 Student Information Systems.....	7
2.1.2 Academic Performance Tracking.....	7
2.2 Theoretical Framework.....	8
2.3 Previous Studies and Related Works.....	9
2.3.1 MyStudyLife.....	9
2.3.2 Todoist and general productivity tools.....	9
2.3.3 Grade calculator applications.....	9
2.3.4 Institutional learning management and student information systems....	10
2.3.5 Learning analytics dashboards.....	10
2.4 Summary of Literature Review.....	10
2.5 Conclusion.....	11
CHAPTER THREE: SYSTEM SPECIFICATION AND DESIGN.....	12
3.1 Methodology.....	12
3.1.1 Research Design.....	12
3.1.2 Data Collection Methods.....	12
3.1.3 System Development Model.....	13
3.2 System Specification.....	13
3.2.1 Analysis of Requirements.....	13

3.2.2 Stakeholder Analysis.....	14
3.2.3 Functional Requirements.....	14
3.2.4 Non-functional Requirements.....	15
3.2.5 Use Cases.....	16
3.3 System Design.....	17
3.3.1 Architectural Design.....	17
3.3.2 Database Design.....	18
3.3.3 User Interface Design.....	19
3.3.4 Security Considerations.....	20
3.4 Conclusion.....	21
References.....	22

LIST OF TABLES

Table 3.1 Stakeholder analysis

Table 3.2 Functional requirements of the system

Table 3.3 Non-functional requirements of the system

Table 3.4 Summary of the principal use cases

Table 3.5 Logical data stores and their contents

Table 3.6 Development tools and technologies

LIST OF FIGURES

Figure 3.1 The Waterfall model as applied to the project

Figure 3.2 Use case diagram of the system

Figure 3.3 Three-tier architectural design

Figure 3.4 Entity relationship diagram

Figure 3.5 The dashboard as implemented

LIST OF ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
ERD	Entity Relationship Diagram
FOCIS	Faculty of Computing and Information Systems
GCTU	Ghana Communication Technology University
GPA	Grade Point Average
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LMS	Learning Management System
SAPTS	Student Academic Progress Tracking System
SIS	Student Information System
SRL	Self-Regulated Learning
SUS	System Usability Scale
UI	User Interface
UML	Unified Modeling Language
XAMPP	Cross-platform (X), Apache, MariaDB, PHP and Perl

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to the individuals whose contributions made this project possible. First and foremost, we thank the Almighty God for the wisdom, health and strength granted to us throughout this work.

We are deeply grateful to our project supervisor, MR. FRANK BOATEN, for his expert guidance, his patience in reading successive drafts, and the constructive criticism that shaped both the system and this report. His insistence on rigour improved this work considerably.

Our appreciation extends to the Head and to the faculty members of the Faculty of Computing and Information Systems, Ghana Communication Technology University, for the academic foundation and the resources that made the project feasible.

We also thank the students and lecturers who gave their time to the interviews and to the usability sessions; their candid feedback exposed problems we would otherwise have missed. Finally, we acknowledge our families for their unwavering support, encouragement and patience throughout our academic journey.

CHAPTER ONE

INTRODUCTION

1.1 Background to Study

The contemporary academic environment at institutions such as Ghana Communication Technology University (GCTU) presents students with challenges that extend well beyond the comprehension of course content. In an era characterised by information abundance and by competing demands on student attention, effective self-management has emerged as a decisive determinant of academic success. The digital transformation of education has, paradoxically, both simplified and complicated the student experience: technology provides unprecedented access to learning resources, yet it also fragments attention and produces an information load that undermines systematic academic tracking.

Traditional methods of academic self-regulation prove increasingly inadequate against modular course structures and continuous assessment. Manual grade calculations in physical notebooks, mental tallies of study hours and disjointed goal-setting across different platforms lack integration, forcing students to maintain several disconnected records that never combine into a single view of academic standing. The cognitive effort required to reconcile information drawn from such disparate sources leads predictably to oversight, miscalculation and reactive rather than proactive management of the semester.

The pedagogical shift towards student-centred learning places a correspondingly greater responsibility on the learner to monitor personal progress. Zimmerman (2002) demonstrated that metacognitive awareness, the capacity to reflect upon one's own learning process, correlates strongly with academic achievement, and Zimmerman and Schunk (2011) established that this capacity is developed rather than innate. Most students, however, lack the structural tools with which to develop that awareness systematically. A clear opportunity therefore exists for a technological intervention that bridges educational theory and daily practice.

Within GCTU's specific context as a technology-focused institution, a digital progress tracker represents more than a convenience: it is the practical application of the university's own core competencies to a problem experienced by its own students. By employing modern web technologies such as responsive layout, client-side data persistence and interactive visualisation, this project sets out to transform academic

self-management from a burdensome clerical chore into an insightful and motivating process.

1.2 Statement of the Problem

The central problem addressed by this project is the systematic inefficiency of academic self-monitoring among university students, an inefficiency that produces suboptimal performance, avoidable stress and missed opportunities for timely correction. The problem manifests along four interconnected dimensions.

First, students operate within fragmented data ecosystems. Grades reside on portals or in electronic mail, deadlines in calendar applications, notes in physical books or scattered files, and personal targets in memory. Reconciling these sources imposes a cognitive overhead heavy enough that many students abandon systematic tracking altogether and fall back on intuition, which is unreliable for academic planning.

Second, in the absence of visualised progress metrics, students struggle to form an accurate picture of where they stand. Progress evaluated through isolated numerical marks alone conceals trends, obscures patterns of strength and weakness, and hides incremental improvement. Students consequently misjudge their position within a course, becoming complacent when they are behind or anxious when they are in fact performing adequately.

Third, a temporal disconnect separates effort from outcome. Students may invest substantial hours in study without ever relating that investment to a specific result. Without a means of logging study sessions against particular courses, no return-on-investment analysis of study strategy is possible, and ineffective methods persist unchallenged from one semester to the next.

Fourth, the absence of structured goal-setting and milestone tracking denies students the motivational benefit of perceived progress. Where larger objectives are never broken into manageable tasks, the academic year is experienced as an undifferentiated marathon rather than a series of achievable stages, raising the likelihood of disengagement.

Generic productivity tools do not resolve these difficulties because they lack domain-specific design. They require the student to adapt a general instrument to an academic purpose, adding configuration burden while still failing to supply integrated academic intelligence such as weighted grade projection. This project therefore addresses a clear and identifiable gap: the need for a purpose-built system that

consolidates academic tracking, visualises progress meaningfully, connects effort to outcome, and structures goal achievement for the university learning context.

1.3 Aim and Objectives

The aim of this project is to design, develop and evaluate a web-based Student Academic Progress Tracking System that enables university students to monitor, understand and improve their academic performance from a single integrated interface, deployed on a local Apache server supplied by XAMPP.

In pursuit of this aim, the following specific objectives were set:

1. To analyse the academic tracking needs of university students through a review of the relevant literature, interviews with students and lecturers, and an examination of the tools currently in use.
2. To design an intuitive and responsive user interface that minimises cognitive load while surfacing the information a student most needs to act upon.
3. To develop the core functional modules of the system, namely course management, assessment and grade recording, study session logging, revision through flashcards, document storage, and goal and task tracking.
4. To implement a client-side data persistence layer using the Web Storage API and IndexedDB, so that academic records survive browser sessions without requiring an external database server.
5. To deploy the completed system on the Apache web server provided by XAMPP and to verify that it operates correctly when served over HTTP from the local host.
6. To test the system for functional correctness, usability and performance, and to evaluate the outcome against each of the objectives stated above.

1.4 Significance of the Study

The significance of this study is both practical and academic. Practically, it delivers a working instrument that GCTU students may use without cost, without an account and without an internet connection, addressing a need that the researchers themselves experienced throughout their programme. Because every record remains on the student's own device, the system also sidesteps the privacy concerns that accompany cloud-hosted academic data.

Academically, the study contributes a concrete demonstration that the principles of self-regulated learning described by Zimmerman (2002) can be operationalised in software: each of the three phases of the cycle is represented by a specific module of the system, and the report traces that mapping explicitly. The project further demonstrates the extent to which a modern browser, treated as an application platform in its own right, can support a complete three-tier information system without server-side processing, which is a result of interest to developers working under constrained infrastructure.

For the Faculty, the work provides a documented reference implementation of a client-side academic application, together with the analysis, design models and testing evidence that accompany it, and may therefore serve as a template for subsequent student projects of a similar nature.

1.5 Scope of the Study

The boundaries of this project were drawn deliberately, so that a complete and dependable system could be delivered within the time available while leaving clear room for later expansion.

The following fall within the scope of the project:

- A front-end web application built with HTML5, CSS3 and vanilla JavaScript, with Chart.js as the only external library, served locally by the Apache component of XAMPP.
- Eight integrated functional modules: dashboard, course management, grades and grade point average analysis, study tracking, flashcards, document vault, goals and tasks, and settings.
- Client-side data management using the Web Storage API for structured records and IndexedDB for uploaded files, including validation, error handling and backup export and import in JSON format.
- A responsive interface that adapts to desktop, tablet and mobile displays, with both a light and a dark theme.
- Functional, usability and performance testing with a representative sample of students, and complete project documentation.

The following are excluded from the scope of the project:

- Multi-user infrastructure: the system is a single-user, device-specific tool without authentication, server-side components or synchronisation across devices.
- Integration with institutional systems: no direct connection is made to the university's learning management system or student information system, and all data is entered manually.
- Collaborative or social features such as sharing progress with peers, comparison against class averages, or interaction with lecturers.
- Predictive analytics driven by machine learning; only deterministic projection from weighted averages is implemented.
- Support for legacy browsers. The system targets current versions of Chrome, Edge, Firefox and Safari with full support for ES6 and CSS Grid.

1.6 Organization of the Study

This report follows the structure prescribed by the Faculty of Computing and Information Systems for final-year undergraduate projects, and its five chapters mirror the system development lifecycle.

Chapter One, the present chapter, introduces the study. It establishes the background, states the problem, sets out the aim and objectives, explains the significance and scope of the work, and describes the organisation of the report.

Chapter Two reviews the relevant literature. It examines scholarship on student information systems and academic performance tracking, presents the theoretical framework of self-regulated learning upon which the design rests, analyses previous studies and related systems, and identifies the research gap that the project addresses.

Chapter Three covers system specification and design. It sets out the research methodology and the development model adopted, documents the requirements analysis, stakeholder analysis, functional and non-functional requirements and use cases, and then presents the architectural design, data design, interface design and security considerations of the system.

Chapter Four reports the implementation of the system. It describes the programming languages and tools used, explains how the principal modules were constructed and deployed under XAMPP, and presents the testing and quality assurance activities together with their results.

Chapter Five concludes the report. It discusses the completed system in comparison with existing solutions, records the challenges encountered and the lessons drawn from them, states the contribution of the work, and makes recommendations for future development.

1.7 Conclusion

This chapter has established that university students lack an integrated means of monitoring their academic progress, and that the fragmented tools presently in use fail them along four specific dimensions: dispersed data, absent visualisation, an unmeasured relationship between effort and outcome, and unstructured goal-setting. The aim and six objectives set out in section 1.3 define the response to that problem, and the scope stated in section 1.5 bounds it to what can be delivered and defended. The chapter that follows examines the body of literature and the existing systems against which this response must be justified.

CHAPTER TWO

LITERATURE REVIEW

2.1 Overview of Relevant Literature

This chapter examines the theoretical foundations, the existing solutions and the technological frameworks that bear upon academic progress tracking. The purpose of the review is to establish what is already known, what tools are already available, and which gaps remain open and therefore justify the development of the proposed system.

2.1.1 Student Information Systems

Student Information Systems are computer-based platforms used by educational institutions to manage student data, including academic records, attendance registers, fee accounts and personal information. Bharamagoudar, Geeta and Totad (2013) describe the migration of such systems onto the web as a decisive improvement in the accessibility and accuracy of institutional records, since a single authoritative store replaces the duplicated ledgers that preceded it. Ellis (2009) draws the related distinction between a student information system, which is administrative in purpose, and a learning management system, which is instructional; both are institution-facing.

That orientation is precisely the limitation relevant to this project. Institutional systems are designed to serve the registry rather than the learner. They publish a final mark once an assessment has been ratified, but they do not help a student to reason about a mark before it is earned, to project the result of a pending examination, or to relate a disappointing score to the hours that preceded it. Sarker, Davis and Tiropanis (2010), reviewing how higher education institutions use student data, observe that the overwhelming majority of that use is administrative and that comparatively little of it is returned to students in a form they can act upon.

2.1.2 Academic Performance Tracking

Academic performance tracking is the continuous assessment and evaluation of learning outcomes for the purpose of identifying strengths and weaknesses while there is still time to act. Black and Wiliam (1998), in their influential review of formative assessment, established that frequent low-stakes feedback produces learning gains

substantially larger than those obtained from summative examination alone. Hattie and Timperley (2007) refined this finding, showing that feedback is most effective when it addresses the gap between current performance and a defined goal, and least effective when it consists of a bare grade.

The implication for system design is direct. A tracker that reports only a mark reproduces the least effective form of feedback. A tracker that situates that mark against a target, a trend and a record of effort supplies the three elements Hattie and Timperley identify as necessary, namely where the learner is going, how the learner is progressing, and what should be done next. Siemens and Long (2011) extend the same logic to learning analytics, arguing that the value of educational data lies in its timely return to the learner rather than in its accumulation.

2.2 Theoretical Framework

The system rests upon the theory of Self-Regulated Learning developed by Zimmerman (2002), which models effective learning as a cycle of three phases. The design of the system maps each phase onto a specific group of features, and this mapping is the principal justification for the module structure presented in Chapter Three.

In the forethought phase the learner analyses the task, sets goals and plans a strategy. The system supports this phase through its goals and tasks module, in which academic objectives are recorded with due dates and priorities, and through course management, in which the assessment structure of each course is declared in advance so that the weight of what is coming is visible before it arrives.

In the performance phase the learner executes the plan while monitoring attention and progress. The system supports this phase through the study tracker, which times focused work using the interval technique described by Cirillo (2018) and logs each completed session against a course, and through the flashcard module, which applies the retrieval practice shown by Roediger and Karpicke (2006) to produce durable retention superior to repeated re-reading.

In the reflection phase the learner evaluates the outcome and attributes it to a cause. The system supports this phase through grade analysis and the dashboard, which display the weighted average and grade point average, the distribution of letter grades and the relationship between weekly study hours and performance, so that attribution rests on recorded evidence rather than on recollection.

Two further bodies of theory inform the design. Pintrich (2004) supplies the framework by which motivation and self-regulation are assessed jointly, supporting the decision to display progress towards a weekly study goal alongside academic results. Dunlosky and colleagues (2013), in their systematic evaluation of learning techniques, rank distributed practice and practice testing as the two most effective strategies available to students, and both are directly represented in the system's study and flashcard modules.

2.3 Previous Studies and Related Works

Five categories of existing solution were examined. Each was assessed against four criteria drawn from the problem statement: integration of academic functions, visualisation connecting effort to outcome, academic specificity, and accessibility in terms of cost and connectivity.

2.3.1 MyStudyLife

MyStudyLife is a cross-platform student planner offering timetable management, homework tracking and examination reminders. Its scheduling model is genuinely academic, which distinguishes it from general planners, and synchronisation across devices is handled well. It nevertheless stops at scheduling: the application records that an assessment exists and when it is due, but not what was scored, and it therefore offers neither weighted grade computation nor any analysis of performance over time. It also requires an account and a network connection for its principal features.

2.3.2 Todoist and general productivity tools

Todoist, and comparable task managers such as Microsoft To Do and Notion, provide mature task capture, recurring reminders and project hierarchies. They are, however, domain-neutral by design. A student wishing to track a course must first construct an academic structure out of generic projects and labels, and even then the tool has no concept of credit hours, assessment weighting or grade points, so no academic calculation can be performed. The configuration burden falls entirely on the student, and the analytical benefit remains unavailable.

2.3.3 Grade calculator applications

A large class of web and mobile grade calculators performs exactly one function: given a set of component marks and their weights, it returns a course average or the mark required in a final examination. These tools are academically specific and computationally correct, and many are free. Their limitation is that they are stateless. Figures are typed in, a result is read off, and nothing is retained, so no history accumulates, no trend can be observed, and the calculation must be repeated in full whenever a new mark is received.

2.3.4 Institutional learning management and student information systems

Platforms such as Moodle, Blackboard and the portals operated by individual universities hold authoritative records and are the natural source of truth for a student's marks. As established in section 2.1.1, however, their orientation is administrative. Marks appear only after ratification, the presentation is tabular rather than analytical, study effort is not represented at all, and the student has no ability to model a hypothetical outcome. Access additionally depends on institutional credentials and on network availability, both of which fail at inconvenient moments.

2.3.5 Learning analytics dashboards

Course Signals, developed at Purdue University and reported by Arnold and Pistilli (2012), is the best-documented example of a learning analytics dashboard that returns predictive information to students, using a traffic-light indicator to signal risk of failure. Reported improvements in retention were substantial. Such systems demonstrate the value of returning analysis to the learner, but they are institutional deployments requiring integration with enrolment and activity data, significant infrastructure and institutional consent. They are not available to an individual student, and no equivalent exists at GCTU.

2.4 Summary of Literature Review

The literature establishes a consistent case for computerised academic tracking. The benefits reported across the studies reviewed above are improved accuracy of data, since calculation is performed rather than remembered; faster generation of reports and summaries; secure and durable storage of records; and convenient access to a complete academic history. Where visualisation accompanies these benefits, the literature

further reports gains in motivation and in metacognitive awareness, because a trend that is seen is understood more readily than a column of figures.

The review of related systems, however, identifies three deficiencies that persist across every category examined. The first is a lack of integration: scheduling, grade calculation, effort logging and revision are each served by a different tool, and students must therefore operate several applications that do not communicate, which reproduces at the level of software the fragmentation described in section 1.2. The second is the absence of visualisation connecting effort to outcome: no tool examined records study time and academic results in the same data set, so no tool can display the relationship between them. The third is a lack of simple, free and academically specific design: the tools that are academically specific are narrow, and the tools that are broad are not academically specific.

The research gap follows directly from these findings. No existing tool combines client-side privacy, zero-cost accessibility that does not depend on a network connection, an academically specific workflow encompassing credit hours and weighted assessment, and comprehensive visual analytics, within a single integrated system. It is this combination that the present project sets out to provide.

2.5 Conclusion

This chapter has reviewed the literature on student information systems and academic performance tracking, established self-regulated learning as the theoretical framework for the design, and assessed five categories of existing system against the criteria derived from the problem statement. The review demonstrates that the proposed system is theoretically grounded in an established model of learning, that it is technologically justified by the capabilities of the modern browser, and that it is practically necessary because the gap it addresses remains open. The chapter that follows translates this justification into a specification and a design.

CHAPTER THREE

SYSTEM SPECIFICATION AND DESIGN

3.1 Methodology

This section sets out the research design adopted for the project, the methods by which requirements were gathered, and the software development model that governed the sequence of work.

3.1.1 Research Design

The project adopts a system development research design. Rather than testing a hypothesis by experiment, the study answers its research question by constructing an artefact and evaluating it against stated requirements. The design proceeds through analysis, specification, construction and evaluation, and the evidence offered for the claims made in Chapter Five is the measured behaviour of the completed system.

3.1.2 Data Collection Methods

Three methods were used to gather the information from which the requirements were derived:

- Interviews. Semi-structured interviews were conducted with students across several levels of study and with lecturers responsible for continuous assessment, in order to establish how marks are currently recorded, what students find difficult to determine about their own standing, and which calculations they perform by hand.
- Observation. The record-keeping practices actually in use were observed directly, including notebooks, spreadsheet files and calculator applications, so that the description of the existing arrangement rests on what students do rather than on what they report doing.
- Review of existing systems and records. The tools examined in section 2.3 were used in practice, and published academic records and assessment structures were reviewed, to establish the data a system of this kind must hold and the calculations it must perform.

The findings from these three sources were consolidated into the requirements presented in section 3.2, and any requirement supported by only one source was verified against the others before it was accepted.

3.1.3 System Development Model

The Waterfall model, first described by Royce (1970) and treated as a standard reference model by Sommerville (2016), was selected to govern the development. The choice was made on three grounds. The requirements of the system were well understood at the outset and were not expected to change materially, which is the condition under which a sequential model performs well. The project operated to a fixed academic timetable with defined deliverables, which a phase-based model supports directly. And the documentation produced at the end of each phase corresponds to the structure required of this report.

The five phases were applied as shown in Figure 3.1. Requirements analysis produced the specification in section 3.2; system design produced the models in section 3.3; implementation produced the working application and its deployment under XAMPP; testing verified the result against the frozen requirements; and deployment and maintenance covered installation, user documentation and corrective work arising from testing.

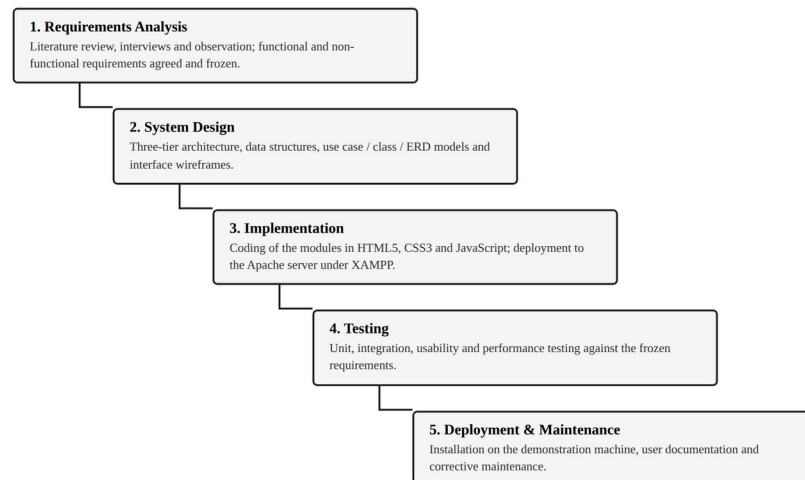


Figure 3.1: The Waterfall model as applied to the project

3.2 System Specification

3.2.1 Analysis of Requirements

Analysis of the material gathered by the methods described above produced a consistent account of the existing arrangement. Students maintain course details in one place, marks in another and study intentions in a third; averages are computed by hand when they are computed at all; credit-weighted grade point averages are typically estimated rather than calculated; and study time is not recorded in any systematic form.

No participant was able to state, without recalculation, the current weighted standing in a named course.

From this account the required capabilities of the system follow. The system must hold courses with their credit hours, hold assessments with their weights and scores, calculate weighted averages and the grade point average automatically, record study sessions against courses, allow goals and tasks to be tracked to completion, present the resulting data visually, and retain everything across browser sessions without a server. These capabilities are stated formally in the two sections that follow.

3.2.2 Stakeholder Analysis

Four groups hold an interest in the system. Their concerns, and the way the design responds to each, are summarised in Table 3.1.

Table 3.1: Stakeholder analysis

Stakeholder	Interest in the system	How the design responds
Student (primary user)	An accurate, immediate view of academic standing; a record of effort; timely warning of what is due	All eight modules; the dashboard aggregates standing, effort and outstanding work on one screen
Lecturer	Students who arrive at consultation with an informed account of their own performance	Grade analysis and per-course status provide evidence a student can bring to a consultation
Faculty / Department	A demonstrable final-year artefact; no burden on institutional infrastructure	The system is self-contained and runs on a local Apache server with no institutional integration
Project team	A system deliverable within the academic timetable and defensible on technical grounds	Scope bounded as in section 1.5; a documented three-tier architecture

3.2.3 Functional Requirements

The functional requirements state what the system does. Each is identified so that the testing reported in Chapter Four may be traced back to it.

Table 3.2: Functional requirements of the system

ID	Requirement	Description
FR1	Manage courses	The user can add, amend and delete courses, each with a name, code, credit hours, lecturer and semester, and can search, filter and sort the resulting list.
FR2	Record assessments with weights	The user can enter assessment components such as quizzes, assignments and examinations, each with a

ID	Requirement	Description
		percentage weight and a score, reflecting actual assessment policy.
FR3	Calculate averages automatically	On entry of a score the system recalculates the weighted course average and the credit-weighted grade point average on the 4.0 scale, removing manual calculation error.
FR4	Log and categorise study sessions	The user can time a focused study session or enter one manually, categorise it by course, and review the accumulated history by day and by week.
FR5	Manage goals and tasks	The user can create tasks with due dates and priorities, group them as overdue, due today or upcoming, and mark them complete against a progress indicator.
FR6	Revise using flashcards	The user can create decks and cards, study them in a flip-to-reveal mode, and mark cards as mastered so that revision concentrates on weaker material.
FR7	Store documents in a vault	The user can write notes and upload images or PDF files such as past questions, and retrieve them against a named course.
FR8	Generate visual progress reports	The system renders charts of grade progress by course, letter grade distribution and study hours over the preceding seven days.
FR9	Persist data across sessions	All records survive the closing of the browser and the restarting of the machine, so that no information is lost between sessions.
FR10	Export and import a backup	The user can export all structured data to a JSON file and restore it later, permitting transfer between machines and recovery after data is cleared.

3.2.4 Non-functional Requirements

The non-functional requirements state how well the system performs its functions. Each is expressed so that it can be verified by measurement rather than by opinion.

Table 3.3: Non-functional requirements of the system

ID	Requirement	Criterion for acceptance
NFR 1	Responsive design	The interface adapts without horizontal scrolling to viewport widths from 360 pixels to 1920 pixels, covering phone, tablet and desktop displays.
NFR 2	Performance	The application becomes interactive within three seconds on the target hardware, and recalculation after a score is entered completes without perceptible delay.
NFR 3	Usability	A student unfamiliar with the system can complete the core tasks of adding a course, entering a score and reading

ID	Requirement	Criterion for acceptance
		the resulting average within ten minutes and without training.
NFR 4	Data security and privacy	All data is held in the browser on the user's own device; no academic record is transmitted to any remote server at any point.
NFR 5	Reliability	Invalid input is rejected with an explanatory message rather than corrupting stored data, and a failure to read one record does not prevent the remainder from loading.
NFR 6	Availability offline	The system operates with the network disconnected, all fonts, icons and libraries being served from the local installation.
NFR 7	Maintainability	The code is organised into named modules with a single storage layer, so that a change to persistence affects one region of the source only.

3.2.5 Use Cases

The primary actor of the system is the Student, who is also the sole actor: because the system is single-user and requires no authentication, there is no administrator or lecturer role. The principal use cases are shown in Figure 3.2 and described in Table 3.4.

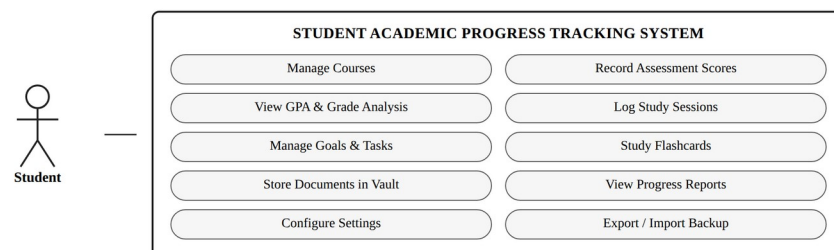


Figure 3.2: Use case diagram of the system

Table 3.4: Summary of the principal use cases

Use case	Precondition	Main flow	Postcondition
Manage Courses	The system is open	The student opens My Courses, supplies the course details and saves	The course is stored and appears in all course selectors
Record Assessment Scores	At least one course exists	The student selects the course, enters the component, its weight and the score, and saves	The score is stored and the course average is recalculated
View GPA and Grade	At least one graded course	The student opens Grades and GPA and reads the average, the	No change to stored data; the

Use case	Precondition	Main flow	Postcondition
Analysis	exists	grade point average and the distribution chart	analysis is displayed
Log Study Sessions	The system is open	The student starts the timer against a course, works, and the completed interval is logged automatically	The session is stored and the weekly total and streak are updated
Manage Goals and Tasks	The system is open	The student creates a task with a due date and priority, and later marks it complete	The task moves group and the progress indicator advances
Export or Import Backup	The system is open	The student exports the data to a JSON file, or selects a file to restore and confirms	A backup file is written, or the stored data is replaced by the backup

3.3 System Design

3.3.1 Architectural Design

The system follows a three-tier architecture, shown in Figure 3.3, in which the three tiers are logically separate but all execute on the client. This arrangement preserves the separation of concerns that the pattern exists to provide, while removing the server-side processing that the privacy and accessibility requirements rule out.

The presentation tier consists of the HTML5 document and the CSS3 stylesheet. It is responsible for structure and appearance only: layout is achieved with CSS Grid and Flexbox, the theme is expressed as a set of custom properties so that the light and dark schemes differ only in their values, and no business rule is encoded at this level.

The application logic tier consists of the JavaScript modules. It holds the router that exchanges one view for another without a page reload, the grade calculator that derives weighted averages and grade points, the study session engine that implements the interval timer, the task manager, the chart renderer that drives Chart.js, and the validation and backup service. This tier is the only part of the system that computes.

The data tier consists of the browser's own storage facilities, reached exclusively through a single storage layer in the source so that no other module addresses storage directly. Structured records are held in the Web Storage API as

JSON documents; binary uploads are held in IndexedDB, which unlike Web Storage accommodates files of substantial size.

Apache, supplied by the XAMPP distribution, serves the three tiers to the browser over HTTP from the local host. It performs no application processing; its role is confined to delivering the files, applying the MIME types and security headers declared in the configuration, and thereby reproducing the conditions of a genuine deployment.

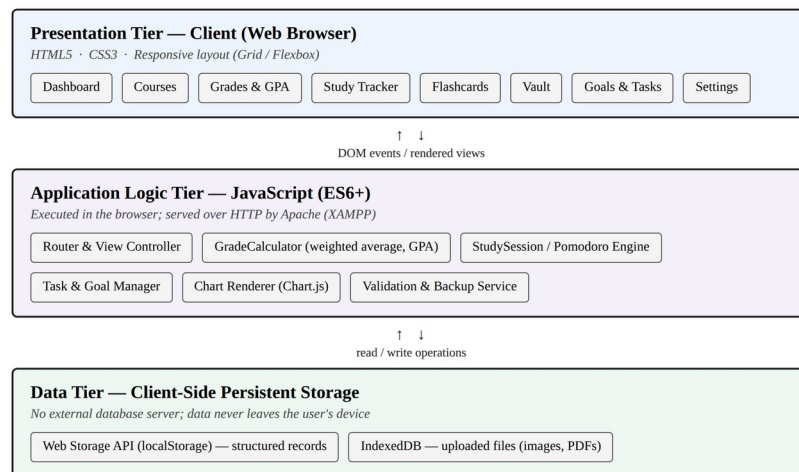


Figure 3.3: Three-tier architectural design

3.3.2 Database Design

The system stores its data on the client rather than in a relational database server, but the data is nonetheless modelled relationally, and the entity relationship diagram in Figure 3.4 governs how records reference one another. Each logical entity is persisted as a JSON collection under its own key, and the relationships between entities are maintained by identifier references in exactly the manner a foreign key would enforce.

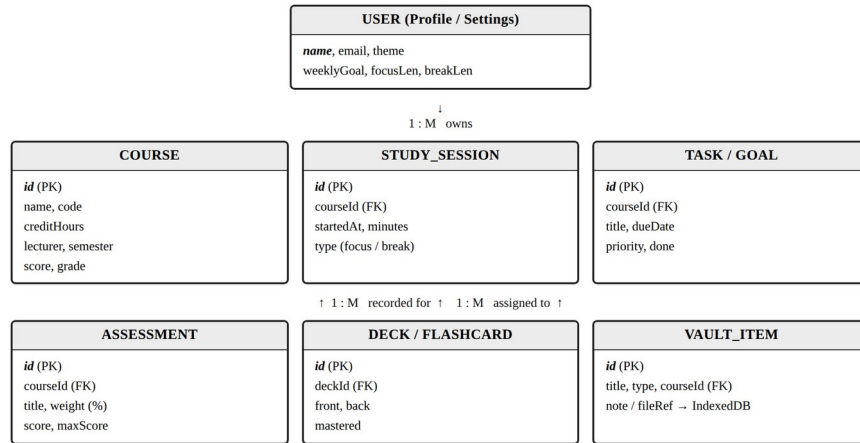


Figure 3.4: Entity relationship diagram

The cardinalities are as follows. One user owns many courses. One course has many assessments, many study sessions and many tasks. One deck has many flashcards. A vault item may optionally reference a course, and where it carries an uploaded file it holds a reference to the record in IndexedDB rather than the file itself, so that the structured collection remains small and quick to parse. Table 3.5 lists the stores and their contents.

Table 3.5: Logical data stores and their contents

Store	Medium	Principal fields
Courses	Web Storage	id, name, code, creditHours, lecturer, semester, score, grade
Assessments	Web Storage	id, courseId, title, weight, score, maxScore
Study sessions	Web Storage	id, courseId, startedAt, minutes, type
Tasks and goals	Web Storage	id, courseId, title, dueDate, priority, done
Decks and cards	Web Storage	id, deckId, front, back, mastered
Vault items	Web Storage	id, title, type, courseId, note, fileRef
Uploaded files	IndexedDB	id, blob
Settings	Web Storage	name, email, theme, weeklyGoal, focusLen, breakLen

3.3.3 User Interface Design

The interface was designed against the usability heuristics of Nielsen (1994) and the guidance of Shneiderman and Plaisant (2010), and its visual presentation of quantities follows the principles set out by Few (2012). Four decisions shaped the result.

The first is a persistent left sidebar carrying the eight modules, so that the structure of the system is visible at all times and the current location within it is always marked. The second is a dashboard that answers the question a student actually arrives with, namely how matters stand, by placing the grade point average, task completion, weekly study hours and the current streak on a single row of summary tiles above the charts.

The third is a consistent flat visual treatment in which colour carries meaning rather than decoration: a fixed palette maps each letter grade to one colour, and that mapping is used identically in the badges, the charts and the status indicators, so that a colour learned in one view is understood in every other. The fourth is a responsive layout that collapses the sidebar to an icon rail below 768 pixels and reflows the tiles and charts into a single column, together with a light and a dark theme selectable in settings. Figure 3.5 shows the dashboard as implemented.

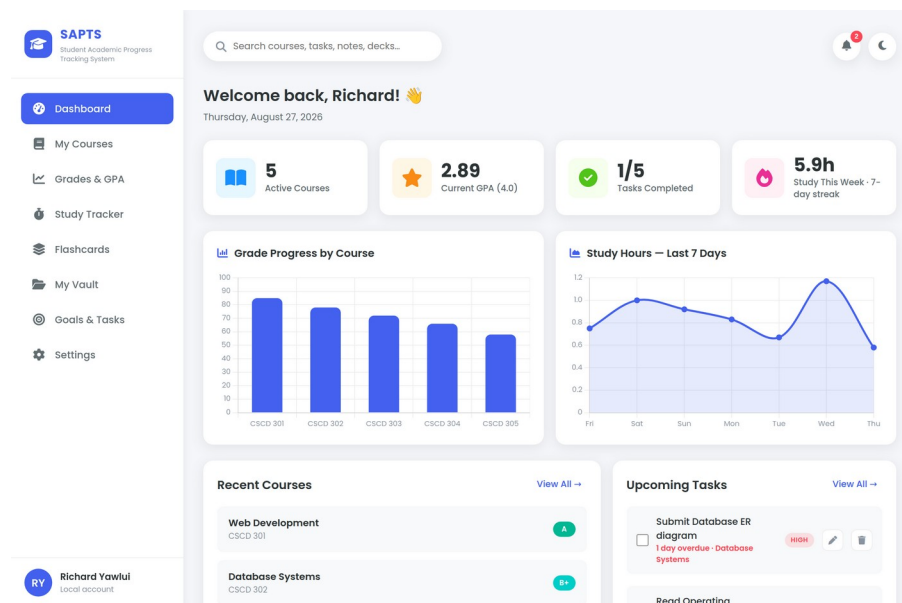


Figure 3.5: The dashboard as implemented

3.3.4 Security Considerations

The security posture of the system follows from its architecture. Because no academic data is transmitted, the entire class of risks associated with data in transit and with breach of a central store does not arise: there is no server to compromise and no database of student records to exfiltrate. The measures that remain relevant are those appropriate to a local application.

- Data confinement. All records are held within the browser's origin-scoped storage on the user's own device, and the system makes no network request for any purpose after the files have been served.
- Input validation. Every value entered is validated for type and range before it is stored, so that malformed input is rejected at the point of entry rather than corrupting the stored collection.
- Output encoding. All user-supplied text is escaped before it is written into the document, which prevents stored cross-site scripting through a course name, a task title or a note.
- Transport and framing controls. The Apache configuration supplied with the system sets X-Frame-Options, X-Content-Type-Options and a referrer policy, and declares explicit MIME types so that no file is interpreted as a type other than its own.
- Availability of data. Because the browser's storage may be cleared by the user or by the operating system, the system provides an export facility, and the user is advised to keep a JSON backup; this is the recovery mechanism in place of server-side redundancy.
- Residual risk. Anyone with access to the unlocked device has access to the data, since the system holds no password. This is a deliberate consequence of the decision to require no account, and it is mitigated by the device's own screen lock rather than by the application.

3.4 Conclusion

This chapter has set out the methodology by which the project was conducted, the requirements the system must satisfy, and the design by which those requirements are met. The research design is constructive, the requirements were drawn from interviews, observation and a review of existing systems, and the Waterfall model governed the sequence of work. Ten functional and seven non-functional requirements were specified and traced to the use cases of the single actor. The design that answers them is a three-tier architecture executing wholly on the client, with a relationally modelled data store held in Web Storage and IndexedDB, an interface built on established usability principles, and a security posture appropriate to a local application. Chapter Four reports the implementation of this design and the testing to which it was subjected.

REFERENCES

- Arnold, KE and Pistilli, MD (2012) 'Course Signals at Purdue: using learning analytics to increase student success', Proceedings of the 2nd International Conference on Learning Analytics and Knowledge, 267-270
- Bharamagoudar, SR, Geeta, RB and Totad, SG (2013) 'Web based student information management system', International Journal of Advanced Research in Computer and Communication Engineering, 2:6, 2342-2348
- Black, P and Wiliam, D (1998) 'Assessment and classroom learning', Assessment in Education: Principles, Policy and Practice, 5:1, 7-74
- Brooke, J (1996) 'SUS: a quick and dirty usability scale', in Jordan, PW et al (eds) Usability Evaluation in Industry, Taylor and Francis, London, 189-194
- Cirillo, F (2018) The Pomodoro Technique: The Life-Changing Time Management System, Virgin Books, London
- Dunlosky, J, Rawson, KA, Marsh, EJ, Nathan, MJ and Willingham, DT (2013) 'Improving students' learning with effective learning techniques', Psychological Science in the Public Interest, 14:1, 4-58
- Ellis, RK (2009) Field Guide to Learning Management Systems, American Society for Training and Development, Alexandria
- Few, S (2012) Show Me the Numbers: Designing Tables and Graphs to Enlighten (2nd edn), Analytics Press, Burlingame
- Hattie, J and Timperley, H (2007) 'The power of feedback', Review of Educational Research, 77:1, 81-112
- Kruchten, P (1995) 'Architectural blueprints: the 4+1 view model of software architecture', IEEE Software, 12:6, 42-50
- Mozilla Developer Network (2024) IndexedDB API [online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API [accessed 12 August 2026]
- Mozilla Developer Network (2024) Web Storage API [online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API [accessed 12 August 2026]
- Nielsen, J (1994) Usability Engineering, Morgan Kaufmann, San Francisco
- Pintrich, PR (2004) 'A conceptual framework for assessing motivation and self-regulated learning in college students', Educational Psychology Review, 16:4, 385-407
- Pressman, RS and Maxim, BR (2020) Software Engineering: A Practitioner's Approach (9th edn), McGraw-Hill, New York
- Roediger, HL and Karpicke, JD (2006) 'Test-enhanced learning: taking memory tests improves long-term retention', Psychological Science, 17:3, 249-255
- Royce, WW (1970) 'Managing the development of large software systems', Proceedings of IEEE WESCON, 1-9

- Sarker, F, Davis, H and Tiropanis, T (2010) 'A review of higher education student data use for learning and teaching', Proceedings of the International Conference on Computer Supported Education, 1-8
- Shneiderman, B and Plaisant, C (2010) Designing the User Interface: Strategies for Effective Human-Computer Interaction (5th edn), Addison-Wesley, Boston
- Siemens, G and Long, P (2011) 'Penetrating the fog: analytics in learning and education', EDUCAUSE Review, 46:5, 30-40
- Sommerville, I (2016) Software Engineering (10th edn), Pearson Education, Harlow
- The Apache Friends (2024) XAMPP Apache + MariaDB + PHP + Perl [online]. Available: <https://www.apachefriends.org> [accessed 12 August 2026]
- Zimmerman, BJ (2002) 'Becoming a self-regulated learner: an overview', Theory Into Practice, 41:2, 64-70
- Zimmerman, BJ and Schunk, DH (2011) Handbook of Self-Regulation of Learning and Performance, Routledge, New York